

# SmartShield IPS

## Step-by-Step Running Guide

### For Linux + Docker • Demo Recording Edition

*Written in plain language — no prior experience needed*

#### What you will do in this guide:

Prepare → Run → Simulate Attacks → See Alerts → Record Demo

# What Is SmartShield?

Think of SmartShield like a smart security guard for a factory. In a real factory, machines talk to each other through a network — PLCs (the computers that control motors, valves, conveyors) send messages back and forth constantly. SmartShield sits quietly and watches all these messages. If someone tries to attack the machines — like sending fake commands, flooding the network, or trying to stop a PLC — SmartShield catches it, logs it, and can even block the attacker.

Your system has two parts:

- **SmartShield IPS:** The brain. Runs as a Python program inside Docker.
- **Web interface:** A webpage you open in your browser to see alerts and what is happening in real time.

## i For your demo

You will run SmartShield on your own Linux PC, pretend your PC is a factory network, simulate attacks (using simple scripts), and show the alerts appearing live on the web interface — exactly what a doctor wants to see.

# What You Need Before Starting

## Check your Linux has these installed

Open a terminal and run each command below. If it prints a version number, you are good. If it says "command not found", install it.

```
# Check Docker
docker --version
# Should show something like: Docker version 24.0.5

# Check Docker Compose
docker compose version
# Should show something like: Docker Compose version v2.20.0

# Check Python
python3 --version
# Should show: Python 3.10 or higher

# Check Git (optional but useful)
git --version
```

## If Docker is not installed:

```
sudo apt update
sudo apt install -y docker.io docker-compose-plugin
sudo systemctl start docker
sudo systemctl enable docker

# Allow your user to run Docker without sudo
```

```
sudo usermod -aG docker $USER
# IMPORTANT: Log out and log back in after this command
```

### If Python is missing:

```
sudo apt install -y python3 python3-pip
```

## Install the attack simulation tools

To make a good demo, you need tools that can send fake attack traffic. Install them:

```
sudo apt install -y nmap hping3 netcat-openbsd

# Install Python Modbus simulator (to simulate PLC attacks)
pip3 install pymodbus scapy --break-system-packages
```



### What these tools do

nmap — scans ports, like a burglar checking which windows are open in a house

hping3 — sends many packets very fast, like flooding a factory gate with trucks

pymodbus — speaks the Modbus language that PLCs use, so we can fake PLC commands

netcat — sends raw data to any port, like knocking on a door very fast

# Steps to Run SmartShield

## STEP 1 Get the Project Code Ready

You already have the zip file (SmartShield-Factory.zip) from the previous session. Put it somewhere easy to find, like your home folder.

Open a terminal and run:

```
# Go to your home folder
cd ~

# If you have the zip file, extract it:
unzip SmartShield-Factory.zip

# Go into the project folder
cd SmartShield-Factory

# See what is inside
ls

# You should see folders like: modules/, config/, docker/, webinterface/, etc.
```

### ✓ Good sign

You should see a list of folders. If you see "modules config docker webinterface zeek-scripts" you are in the right place.

## STEP 2 Set Up the Configuration Files

You need to tell SmartShield which IPs to protect and which are authorized.

*For your demo on a local PC, we will use fake local IP addresses.*

### 2a. Create a simple OT devices protection list:

```
# Open the file with a text editor
nano config/ot_protected_devices.conf

# Add these lines (pretend these are your factory PLCs):
# (Press Ctrl+O to save, Ctrl+X to exit nano)

# For demo: protect these fake PLC IPs
172.20.0.10 # Fake PLC-01
172.20.0.11 # Fake PLC-02
172.20.0.20 # Fake HMI
```

### 2b. Leave the Modbus masters file empty for now

An empty file means SmartShield will detect ALL Modbus writes as unauthorized — perfect for showing attacks in the demo.

```
# Just make sure the file exists (it already does)
ls config/modbus_authorized_masters.conf
```

## 2c. Set your web interface password:

```
# Create the .env file that Docker will use
cat > docker/.env << 'EOF'
SMARTSHIELD_INTERFACE=eth0
SMARTSHIELD_WEB_USER=admin
SMARTSHIELD_WEB_PASSWORD=Demo1234567!
SMARTSHIELD_SECRET_KEY=abc123def456ghi789jkl012mno345pqr678
EOF

echo "Done! .env file created."
```

### ⚠ Important for the demo

The interface name (eth0) does not matter much for a local demo — SmartShield will read from PCAP or network.

For live capture on your machine, replace eth0 with your actual network interface name.

To find your interface name, run: `ip link show`

### STEP 3 Build the Docker Image

This turns the code into a ready-to-run container. You only do this once (or when code changes).

*This may take 5–15 minutes the first time. Go make tea. ☕*

```
# Make sure you are in the project root folder
cd ~/SmartShield-Factory

# Build the image (this takes a while first time)
docker build -f docker/Dockerfile -t smartshield_gp:latest .

# When it finishes you should see:
# Successfully tagged smartshield_gp:latest
```

**If the build fails with a network error:**

```
# Try again — sometimes package downloads timeout
docker build -f docker/Dockerfile -t smartshield_gp:latest . --no-cache
```

**Verify the image was built:**

```
docker images | grep smartshield
# You should see a line like:
# smartshield_gp   latest   abc123...   5 minutes ago   1.2GB
```

### STEP 4 Create the Data Folders

SmartShield needs folders on your computer to save alerts and data between restarts.

```
# Create the folders where SmartShield saves its data
sudo mkdir -p /var/lib/smartshield/redis
sudo mkdir -p /var/lib/smartshield/output

# Give Docker permission to write to them
sudo chmod 777 /var/lib/smartshield/redis
sudo chmod 777 /var/lib/smartshield/output

# Verify they exist
ls /var/lib/smartshield/
# Should show: output redis
```

### STEP 5 Start SmartShield

Now start all the containers with one command. This starts SmartShield, Redis (the database), and the web interface.

```
# Go into the docker folder
cd ~/SmartShield-Factory

# Start everything
docker compose -f docker/docker-compose.factory.yml up -d

# The -d means "run in background" so your terminal stays usable
```

```
# Wait 10 seconds, then check if they are running
sleep 10
docker compose -f docker/docker-compose.factory.yml ps
```

You should see something like this:

| NAME              | STATUS        | PORTS                    |
|-------------------|---------------|--------------------------|
| smartshield       | Up 15 seconds |                          |
| smartshield-redis | Up 15 seconds | 127.0.0.1:6379->6379/tcp |
| smartshield-web   | Up 15 seconds | 0.0.0.0:55000->55000/tcp |

### 💡 If a container shows "Exiting" or "Error"

Run this to see what went wrong:

```
docker logs smartshield
docker logs smartshield-redis
```

The most common problem is the docker/.env file missing — check Step 2.

## STEP 6 Open the Web Interface

The web interface is where you will see all alerts during the demo. Open it in your browser.

```
# Open your browser and go to:
http://localhost:55000

# Or if your Linux has a desktop, run:
xdg-open http://localhost:55000
```

You will see a login page. Enter:

| Field    | Value                                        |
|----------|----------------------------------------------|
| Username | admin                                        |
| Password | Demo1234567! (or whatever you set in Step 2) |

### ✅ You are in!

After login you should see the SmartShield dashboard.

At first it will look empty — that is normal. Alerts appear after attacks are detected.

Keep this browser tab open for the demo.

### To see live logs while it runs (open a second terminal):

```
docker logs smartshield -f
# The -f means "follow" — new lines appear in real time
# Press Ctrl+C to stop watching logs
```

# How to Test It — Simulating Attacks

For your demo, you will run fake attacks on your own computer. SmartShield is watching the network and will detect them. Think of it like a fire drill — you set off the alarm on purpose to prove the system works.

## ⚠ These attacks are safe on your own PC

All the commands below target localhost (127.0.0.1) or your own machine.

NEVER run these on a real factory network or someone else's network.

They are harmless on your own PC and will not damage anything.

## Attack 1 — Port Scan (Simulates a Hacker Discovering the Network)

A port scan is like a burglar walking along a street, knocking on every door to see which ones open. SmartShield will see many connections and flag it.

In a NEW terminal, run:

```
# Simple port scan against your own machine
# This simulates an attacker discovering Modbus port 502
nmap -sS -p 502,102,20000 127.0.0.1

# Aggressive scan — more noise, easier for SmartShield to detect
nmap -sV -p 1-1000 127.0.0.1

# Scan targeting OT ports specifically
nmap -p 502,102,20000,4840,44818,34964 --open 127.0.0.1
```

Then check the web interface — you should see a new alert appear within 1–2 minutes.

## Attack 2 — Modbus Connection Flood (Simulates Flooding a PLC)

This is like someone calling a PLC's phone number thousands of times per minute so the real operator cannot get through. SmartShield will count the connections and alert when the threshold is exceeded.

First, run a Modbus server so there is something to attack:

```
# Terminal 1 — Start a fake Modbus PLC server
python3 - << 'PYEOF'
from pymodbus.server import StartTcpServer
from pymodbus.datastore import ModbusSlaveContext, ModbusServerContext
from pymodbus.datastore import ModbusSequentialDataBlock
import logging
logging.basicConfig()

store = ModbusSlaveContext(
    di=ModbusSequentialDataBlock(0, [17]*100),
    co=ModbusSequentialDataBlock(0, [17]*100),
    hr=ModbusSequentialDataBlock(0, [17]*100),
```



```
    ir=ModbusSequentialDataBlock(0, [17]*100)
)
context = ModbusServerContext(slaves=store, single=True)
print("Fake PLC running on port 502...")
StartTcpServer(context=context, address=("0.0.0.0", 502))
PYEOF
```

Then in a second terminal, run the flood attack:

```
# Terminal 2 - Flood the fake PLC with connections
python3 - << 'PYEOF'
from pymodbus.client import ModbusTcpClient
import time

print("Starting Modbus flood attack...")
for i in range(50):
    try:
        c = ModbusTcpClient("127.0.0.1", port=502)
        c.connect()
        c.read_coils(0, 10)    # Read coils
        c.close()
    except Exception as e:
        pass
    if i % 10 == 0:
        print(f"Sent {i} connections...")

print("Done! Check SmartShield web interface for alerts.")
PYEOF
```



### What to show in the demo

Go to the web interface → Analysis page.

You should see the attacker IP (127.0.0.1) appear with evidence of a Modbus connection flood.  
The threat level will be HIGH.

## Attack 3 — Modbus Unauthorized Write (Simulates Tampering with PLC)

This is the most dangerous attack in real factories. An unauthorized computer sends a WRITE command to a PLC — changing a coil value that could open a valve or stop a motor. SmartShield detects writes from non-authorized sources.

```
# Make sure the fake PLC from Attack 2 is still running in Terminal 1
# In Terminal 2, run an unauthorized write:

python3 - << 'PYEOF'
from pymodbus.client import ModbusTcpClient

print("Attempting unauthorized Modbus coil write...")
client = ModbusTcpClient("127.0.0.1", port=502)

if client.connect():
    # Write Single Coil - function code 5
    # In a real factory this could open a valve or stop a motor
    result = client.write_coil(0, True)
    print(f"Write coil result: {result}")
```

```
# Write Multiple Coils — function code 15
result = client.write_coils(0, [True, False, True, True, False])
print(f"Write multiple coils: {result}")

# Write Single Register — function code 6
result = client.write_register(1, 9999)
print(f"Write register: {result}")

client.close()
print("Attack complete. Check SmartShield!")
else:
    print("Could not connect. Is the fake PLC running?")
PYEOF
```

### What this shows the doctor

SmartShield detects UNAUTHORIZED WRITES to PLCs — the most dangerous ICS attack. The evidence type will be: OT\_MODBUS\_WRITE\_COILS with CRITICAL threat level. In a real factory, this would trigger an alert to the SOC (security operations center).

## Attack 4 — TCP SYN Flood (Simulates DoS on OT Network)

A flood attack overwhelms a PLC or network device so it stops responding. Like calling an emergency phone line 1000 times per second so no one else can call.

```
# Run a SYN flood on port 502 (Modbus) - uses hping3
# This simulates a Denial of Service attack on a PLC

sudo hping3 -S -p 502 --flood 127.0.0.1

# Let it run for 10 seconds, then press Ctrl+C to stop
# SmartShield will detect the flood from the connection count spike
```

## Attack 5 — Quick Nmap Service Scan (Shows Network Discovery)

Attackers first scan to find what is on the network before attacking. This simulates reconnaissance — the first stage of any cyberattack on a factory.

```
# Service version detection scan — loudest type of scan
nmap -sV -O 127.0.0.1

# UDP scan targeting OT protocols
sudo nmap -sU -p 20000,34964,319,123 127.0.0.1

# Aggressive scan with script detection
nmap -A -p 502,102 127.0.0.1
```

### i When to run each attack for the demo

Run attacks in this order for maximum visual impact:

1. Nmap scan first (easy to explain)

2. Modbus flood (dramatic spike in connections)
3. Unauthorized write (most serious — shows CRITICAL alert)
4. Show the web interface filling up with alerts
5. Show blocking.log to prove the attacker was blocked

# Seeing the Results

## In the Web Interface

Go to <http://localhost:55000> in your browser. Here is what each page shows:

| Page                 | What it shows                                                       |
|----------------------|---------------------------------------------------------------------|
| Overview page (/)    | Summary — how many profiles, how many alerts, when analysis started |
| Analysis (/analysis) | The main page — click any IP to see all evidence and attack details |
| Timeline             | A chronological view of all events — great for showing the doctor   |

## In the Terminal

See alerts in real time:

```
# Watch SmartShield logs live
docker logs smartshield -f

# See the alerts file
cat /var/lib/smartshield/output/alerts.json

# Watch new alerts appear (updates every second)
watch -n 1 "cat /var/lib/smartshield/output/alerts.json | tail -20"

# See what IPs are blocked
sudo iptables -L smartshieldBlocking -n 2>/dev/null || echo "blocking module not active"

# Count total alerts generated
docker exec smartshield-redis redis-cli KEYS "alert_*" | wc -l

# List all evidence stored in Redis
docker exec smartshield-redis redis-cli KEYS "evidence_*" | head -20
```

## Quick Status Check Command

Run this any time to see a full health check:

```
# Full status in one command
echo "=== Containers ===" && docker compose -f docker/docker-compose.factory.yml ps
echo "=== Alerts count ===" && docker exec smartshield-redis redis-cli DBSIZE
echo "=== Output files ===" && ls -lh /var/lib/smartshield/output/
echo "=== Redis persisting? ===" && docker exec smartshield-redis redis-cli INFO
persistence | grep aof_enabled
```

# Stopping and Restarting

---

## To stop SmartShield (data is saved)

```
docker compose -f docker/docker-compose.factory.yml down

# All your alerts and data are saved in /var/lib/smartshield/
# Redis writes to disk so nothing is lost
```

## To start again later

```
cd ~/SmartShield-Factory
docker compose -f docker/docker-compose.factory.yml up -d

# Wait 10 seconds then open browser at http://localhost:55000
# Your previous alerts will still be there
```

## To restart (after config changes)

```
docker compose -f docker/docker-compose.factory.yml restart

# Or restart just SmartShield:
docker restart smartshield
```

## To see logs at any time

```
docker logs smartshield --tail 50
docker logs smartshield-redis --tail 20
docker logs smartshield-web --tail 20
```

# Recording a Good Demo for Your Doctor

---

## Recommended Demo Script (8–10 minutes)

| Time      | Section             | What to say/show                                                                                           |
|-----------|---------------------|------------------------------------------------------------------------------------------------------------|
| 0:00–1:00 | Introduction        | Show the web interface (empty, clean). Explain what SmartShield is and where it sits in a factory network. |
| 1:00–2:00 | Start monitoring    | Show the terminal with "docker logs smartshield -f" running. Explain Zeek is capturing traffic.            |
| 2:00–3:30 | Attack 1: Port scan | Run nmap in the terminal. Switch to web interface — show a new IP profile appearing.                       |

| Time       | Section                      | What to say/show                                                                                                  |
|------------|------------------------------|-------------------------------------------------------------------------------------------------------------------|
| 3:30–5:00  | Attack 2: Modbus flood       | Start fake PLC, run flood script. Show connection count rising in web interface.                                  |
| 5:00–7:00  | Attack 3: Unauthorized write | Run write attack. Show CRITICAL alert appearing. Explain this would stop a motor or open a valve in real factory. |
| 7:00–8:00  | Show blocking                | Run: <code>sudo iptables -L smartshieldBlocking -n</code> . Show the attacker IP was blocked.                     |
| 8:00–9:00  | Show OT-safe                 | Explain that if 192.168.10.10 (a PLC) were the source, it would NOT be blocked — factory safe.                    |
| 9:00–10:00 | SIEM export                  | Show alerts.json file. Explain this feeds into enterprise SIEM systems like Splunk or QRadar.                     |

## Screen Recording Tips

- Use OBS Studio or SimpleScreenRecorder on Linux. Install: `sudo apt install obs-studio`
- Record at 1080p if possible. The web interface looks much better in full screen.
- Have TWO windows side by side: terminal on left, browser (web interface) on right.
- Zoom into the alerts section when a CRITICAL alert appears — the red color is very visible.
- Narrate what you are doing: "I am now running a Modbus write attack to coil register 0..."
- Show the timestamp on alerts — it proves detection is real-time, not pre-recorded.

```
# Install screen recorder
sudo apt install simplescreenrecorder

# Or use OBS Studio
sudo apt install obs-studio

# To set up two windows side by side, right-click desktop → Tile Windows
# Or on GNOME: drag one window to the left edge, another to the right edge
```

## What to Say During the Demo

| Moment                        | What to say                                                                                                                                           |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| When showing web UI           | SmartShield uses a behavioral profiling approach — it builds a normal profile for every IP and alerts when behavior deviates from that profile.       |
| When showing Modbus alert     | Modbus is the most common industrial control protocol. An unauthorized write to a coil could open a safety valve or stop an assembly line.            |
| When showing OT-safe blocking | In a real factory, blocking a PLC IP would stop production. SmartShield's OT-safe mechanism prevents this — we alert but never auto-block OT devices. |

| Moment                         | What to say                                                                                                                                         |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| When showing SIEM export       | The syslog export means this integrates with enterprise security systems like Splunk, IBM QRadar, or Wazuh without changing existing SOC workflows. |
| When showing Redis persistence | Even after a server restart, SmartShield remembers all evidence and blocked IPs. Critical for factory environments that run 24/7.                   |

# Common Problems and Fast Fixes

| Problem                                        | Fix                                                                                                                                                                                 |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Permission denied" on Docker                  | Run: <code>sudo usermod -aG docker \$USER</code> — then log out and log in again.                                                                                                   |
| Web interface shows nothing at localhost:55000 | Wait 30 more seconds. Then run: <code>docker ps</code> — check <code>smartshield-web</code> is "Up". Then run: <code>docker logs smartshield-web</code>                             |
| Containers keep restarting / "Exiting"         | Run: <code>docker logs smartshield</code> — look for error messages. Most common: missing <code>.env</code> file in <code>docker/</code> folder.                                    |
| No alerts appearing after attacks              | SmartShield groups evidence into 1-hour time windows. Lower the threshold: edit <code>smartshield.yaml</code> , set <code>evidence_detection_threshold: 0.05</code> , then restart. |
| "Port 502 permission denied"                   | Port 502 needs root. Run: <code>sudo python3 ...</code> for the fake PLC server.                                                                                                    |
| "pymodbus not found"                           | Run: <code>pip3 install pymodbus --break-system-packages</code>                                                                                                                     |
| nmap says "0 open ports"                       | The fake PLC needs to be running first (Terminal 1). Make sure it shows "Fake PLC running on port 502..."                                                                           |
| Alerts.json file is empty                      | SmartShield may need 5–10 minutes to start up fully. Run: <code>docker logs smartshield   grep "Started"</code>                                                                     |



## If all else fails — full reset

```
docker compose -f docker/docker-compose.factory.yml down
sudo rm -rf /var/lib/smartshield/*
sudo mkdir -p /var/lib/smartshield/redis /var/lib/smartshield/output
sudo chmod 777 /var/lib/smartshield/redis /var/lib/smartshield/output
docker compose -f docker/docker-compose.factory.yml up -d
```

# Cheat Sheet — All Commands in Order

Cut this page out and keep it next to your keyboard during the demo.

## Initial Setup (run once)

```
cd ~/SmartShield-Factory
sudo mkdir -p /var/lib/smartshield/{redis,output}
sudo chmod 777 /var/lib/smartshield/{redis,output}
docker build -f docker/Dockerfile -t smartshield_gp:latest .
```

## Start SmartShield

```
cd ~/SmartShield-Factory
```



```
docker compose -f docker/docker-compose.factory.yml up -d
sleep 10 && docker compose -f docker/docker-compose.factory.yml ps
# Then open browser: http://localhost:55000
```

## Demo Attacks (run in order)

```
# Attack 1 — Port scan
nmap -sV -p 502,102,20000 127.0.0.1

# Attack 2 — Start fake PLC (Terminal 1)
sudo python3 -c "
from pymodbus.server import StartTcpServer
from pymodbus.datastore import
ModbusSlaveContext, ModbusServerContext, ModbusSequentialDataBlock
s=ModbusSlaveContext(co=ModbusSequentialDataBlock(0,[17]*100),hr=ModbusSequentialDataBlock(0,[17]*100),ir=ModbusSequentialDataBlock(0,[17]*100),di=ModbusSequentialDataBlock(0,[17]*100))
print("Fake PLC on port 502")
StartTcpServer(ModbusServerContext(slaves=s,single=True),address=("0.0.0.0",502))
"

# Attack 2 — Flood Modbus (Terminal 2)
python3 -c "
from pymodbus.client import ModbusTcpClient
[(__import__("time").sleep(0.1), ModbusTcpClient("127.0.0.1",port=502).connect())
for _ in range(50)]
"

# Attack 3 — Unauthorized write (Terminal 2)
python3 -c "
from pymodbus.client import ModbusTcpClient
c=ModbusTcpClient("127.0.0.1",port=502)
c.connect()
c.write_coil(0,True)
c.write_register(1,9999)
print("Done")
"
```

## Check Results

```
cat /var/lib/smartshield/output/alerts.json
docker logs smartshield --tail 30
sudo iptables -L smartshieldBlocking -n 2>/dev/null
```

## Stop Everything

```
docker compose -f docker/docker-compose.factory.yml down
# Kill the fake PLC: Ctrl+C in Terminal 1
```